



**INSTITUTO SUPERIOR
TECNOLÓGICO QUITO**

Lenguaje de programación 3



Ing. Eduardo Avilés



eduardo.aviles@itq.edu.ec



Back-end con NodeJS y ExpressJS

El backend en MEAN está compuesto principalmente por **Node.js** y **Express.js**, y se encarga de las siguientes funciones clave:

- **Lógica de negocio:** Aquí es donde se implementan las reglas y procesos que dictan cómo funciona tu aplicación. Por ejemplo, si se tiene una aplicación de comercio electrónico, la lógica de negocio en el backend manejaría cosas como calcular precios, procesar pedidos, verificar inventario, etc.

- **Interacción con la base de datos (MongoDB):** El backend es el intermediario entre la aplicación web y la base de datos. Se encarga de guardar, recuperar, actualizar y eliminar datos en **MongoDB** en respuesta a las solicitudes del frontend.
- **API RESTful:** El backend expone una **API (Interfaz de Programación de Aplicaciones)** que el frontend utiliza para comunicarse con él. Esta API generalmente sigue los principios **REST (Representational State Transfer)**, lo que significa que utiliza solicitudes HTTP (GET, POST, PUT, DELETE) para realizar operaciones sobre los recursos.



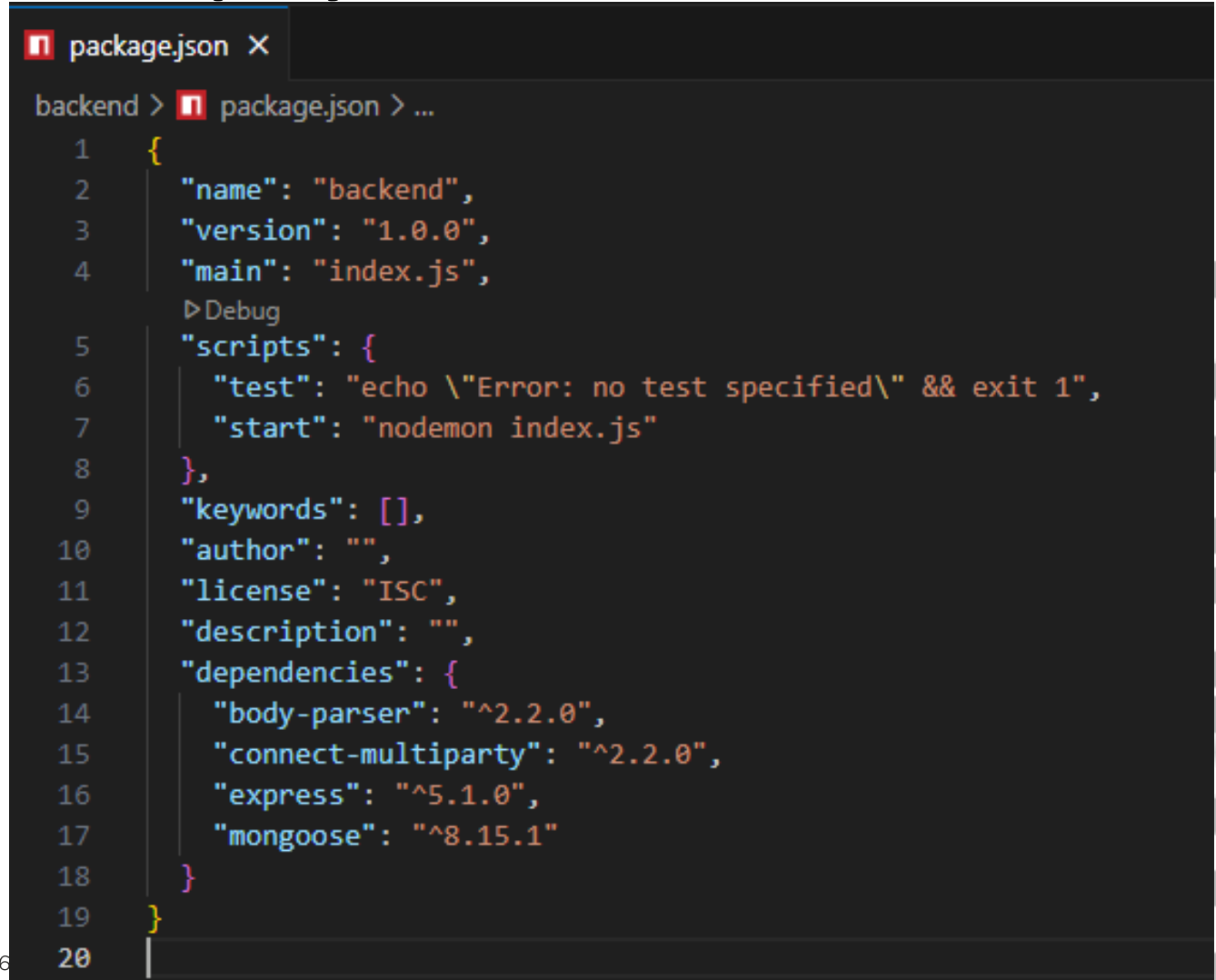
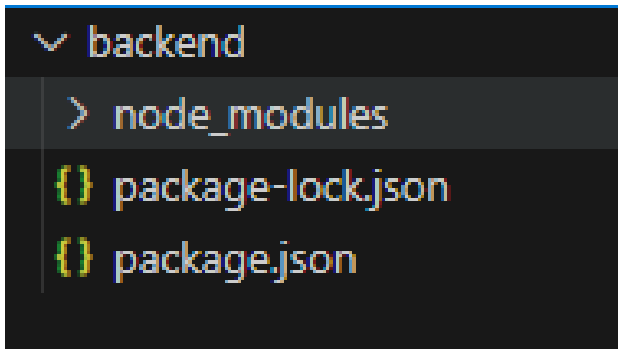
- **Autenticación y autorización:** Se encarga de verificar la identidad de los usuarios (autenticación) y de determinar qué acciones están autorizados a realizar (autorización). Esto es crucial para la seguridad de la aplicación.
- **Manejo de errores:** El backend es responsable de capturar y responder a los errores que puedan ocurrir durante el procesamiento de las solicitudes, enviando mensajes apropiados al frontend.





Comandos para iniciar un nuevo proyecto

- `npm init -y`
- `npm install express`



Dependencias

Body-parser

Este **middleware** se utiliza para analizar los cuerpos de las solicitudes entrantes antes de que lleguen a tus manejadores de ruta, poniéndolos disponibles en la propiedad `req.body`. Es especialmente útil para manejar datos JSON y codificados en URL.

`npm install body-parser`



Nodemon

Es una utilidad que monitorea cualquier cambio en el código fuente y **reinicia automáticamente tu servidor**. Esto es útil para el desarrollo, ya que evita tener que reiniciar manualmente la aplicación de Node.js cada vez que se hace un cambio.

npm install -g nodemon



Connect-multiparty

Este módulo proporciona un middleware de Connect/Express para analizar solicitudes multipart/form-data que se utilizan típicamente para la **subida de archivos**.

npm install connect-multiparty



Mongoose

Mongoose es una biblioteca de **modelado de datos de objetos (ODM)** para MongoDB y Node.js. Proporciona una solución sencilla y basada en esquemas para modelar los datos de tu aplicación y ofrece herramientas poderosas para interactuar con tu base de datos MongoDB.

npm install mongoose





```
package.json X
backend > package.json > main
1  {
2    "name": "backend",
3    "version": "1.0.0",
4    "main": "index.js",
   ▶ Debug
5    "scripts": {
6      "test": "echo \"Error: no test specified\" && exit 1"
7    },
8    "keywords": [],
9    "author": "",
10   "license": "ISC",
11   "description": "",
12   "dependencies": {
13     "body-parser": "^2.2.0",
14     "connect-multiparty": "^2.2.0",
15     "express": "^5.1.0",
16     "mongoose": "^8.15.1"
17   }
18 }
19
```





JS index.js X

backend > JS index.js > ...

```
1  //código más seguro y robusto
2  'use strict'
3  var mongoose=require('mongoose');
4  var port='3600';
5  // promesas nativas de JavaScript
6  mongoose.promise=global.Promise;
7  // importación de la aplicación de Express
8  var app=require('./app');
9  //conexión a la base de datos
10 mongoose.connect('mongodb://localhost:27017/tienda')
11 .then(()=>{
12     console.log("Conexion establecida con la bdd");
13     app.listen(port,()=>{
14         console.log("Conexion establecida en el url: localhost:3600");
15     })
16 })
17 .catch(err=>console.log(err));
```



```
JS app.js X
backend > JS app.js > ...
 1  'use strict'
 2  var express=require('express');
 3  var bodyParser=require('body-parser');
 4
 5  var app=express();
 6  var tienda_routes=require('./routes/tienda');
 7  //todo que lo que llega y se envía se convierta en json
 8  app.use(bodyParser.urlencoded({extended:false}));
 9  app.use(bodyParser.json());
10
11  //configuracion de cabeceras
12  app.use((req,res,next)=>{
13      res.header('Access-Control-Allow-Origin','*');
14      res.header('Access-Control-Allow-Headers',
15          'Authorization, X-API-KEY, X-Request-With,Content-Type,Accept, Access-Control-Allow,Request-Method');
16      res.header('Access-Control-Allow-Methods','GET,POST,OPTIONS,PUT,DELETE');
17      res.header('Allow','GET,POST,OPTIONS,PUT,DELETE');
18      next();
19  });
20
21  app.use('/',tienda_routes);
22
23  module.exports=app;
```



Ing. Eduardo Avilés



eduardo.aviles@itq.edu.ec



0958919713